

Understanding the Impact of Inputs on LLM-Based Automated Test Generation

by

Saarang Agarwal

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Master of Mathematics
in
Computer Science

Waterloo, Ontario, Canada, 2026

© Saarang Agarwal 2026

Author's Declaration

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Abstract

Large Language Models (LLMs) and software development agents are driving a paradigm shift in software engineering, transitioning the field from a human-first to an Artificial Intelligence (AI)-first approach to development. This shift has significantly improved the efficiency of tasks such as code generation and automated testing. However, the performance of these LLMs and agents remains highly dependent on the quality and type of input information they receive. This thesis investigates how different types of input and contextual information influence the effectiveness of LLMs and automated test generation agents through two studies: one focusing on LLMs and the other on test generation agents, each using a distinct dataset.

The first study focuses on LLMs and the impact of different inputs on their ability to generate unit tests, with particular emphasis on the availability of software requirements and the correctness of the code under test. We evaluate five state-of-the-art LLMs of varying scales on the CodeChef dataset and analyze how different input configurations affect the correctness, bug detection capability, and code coverage of the generated unit tests. Our findings indicate that combining code with clear requirements produces the highest-quality test cases, whereas generating tests from incorrect code alone yields significantly poorer results.

The second study focuses on understanding which features of bug reports influence the performance of test generation agents in generating bug-reproducing tests. We identify a set of salient features in bug reports and manually annotate 709 reports from the SWT-Bench Lite and Verified datasets based on the presence of these features. Using selected agents from the SWT-Bench leaderboards, we analyze their test generation capabilities and examine overlaps in the bug reports they resolve. We further leverage the annotated dataset to assess how these features affect agent performance and conduct statistical analyses to determine their relative importance in enabling agents to generate bug-reproducing tests. Our findings indicate that including natural-language solutions, localization information, and descriptions of correct behavior is associated with improved performance in bug-reproducing test generation. Moreover, different agents prioritize different features, reflecting variations in their underlying architectures and LLMs.

Together, these studies provide critical insights into how different types of information influence the effectiveness of LLMs and test generation agents. Readers can use these insights to guide the effective use of current LLMs and agents, as well as to inform the design and development of future systems.

Acknowledgements

I would like to sincerely thank my supervisors, Professor Mei Nagappan and Professor Pengyu Nie, for their guidance and support throughout my time at the University of Waterloo. Your mentorship has not only helped me grow as a researcher but also challenged me to think critically and aim higher than I thought possible. I am deeply grateful for the opportunities you provided me to explore and learn.

I also want to thank all the professors I've had the chance to learn from or work with during my master's. Each of you has contributed to my journey in unique ways, sparking my curiosity and helping me develop a genuine love for research. Special thanks to Professor Chengnian Sun and Professor Yuntian Deng for taking the time to read my thesis and provide insightful feedback that made my work stronger.

I am especially grateful to the members of the Software Analytics Group (SWAG) lab. Your support and sense of humor made the lab a welcoming and inspiring place to be.

Finally, I want to express my heartfelt thanks to my family. To my parents, thank you for your unwavering love, encouragement, and belief in me. Your support has made this journey possible, and I wouldn't be here without you.

Dedication

Dedicated to my family, whose love, support, and encouragement have brought me this far.

Table of Contents

Author’s Declaration	ii
Abstract	iii
Acknowledgements	iv
Dedication	v
List of Figures	ix
List of Tables	xi
1 Introduction	1
1.1 Problem Statement	1
1.2 Thesis Overview	2
2 What Inputs Drive Effective LLM-based Unit Test Generation?	3
2.1 Introduction	3
2.2 Experimental Design	4
2.2.1 Dataset	5
2.2.2 LLM Inputs Construction	5
2.2.3 Test Generation and Reduction	7

2.2.4	Evaluation Metrics	8
2.3	Results	9
2.3.1	Models	9
2.3.2	Test Case Quality	9
2.3.3	Difficulty Level	13
2.3.4	Impact of Model Size	13
2.4	Implications	13
2.5	Conclusion	14
3	What Impacts the Bug-Reproducing Test Generation Capability of Agents?	17
3.1	Introduction	17
3.2	Study Design	18
3.2.1	Benchmark	19
3.2.2	Agents	19
3.2.3	Labeling	20
3.2.4	Research Questions	21
3.3	Analysis and Results	22
3.3.1	RQ1: How effective are current software development agents when generating bug-reproducing tests?	23
3.3.2	RQ2: Which features in bug reports play the most influential role in helping software development agents generate bug-reproducing tests?	25
3.3.3	RQ3: How do bug report features differ between issues uniquely resolved by the top-performing agent and those resolved by other agents but missed by the top-performing agent?	28
3.4	Discussion	30
3.4.1	Suggestions	30
3.4.2	Threats to Validity	31
3.5	Conclusion	32

4 Conclusion	33
4.1 Opportunities for Future Research	34
References	35

List of Figures

2.1	Workflow of our LLM-based test generation evaluation framework. Prompts containing different information form nine different research cases (requirements (R), correct code (CC), first incorrect code (IC ₁), second incorrect code (IC ₂), third incorrect code (IC ₃), requirements and correct code (R-CC), requirements and first incorrect code (R-IC ₁), requirements and second incorrect code (R-IC ₂), and requirements and third incorrect code (R-IC ₃)). These inputs are used to create the user prompt, which along with the system prompt is used by the LLM to generate test cases. These test cases are executed against the correct code to obtain pass/fail and code coverage data. This information is used for test case reduction and obtaining relevant evaluation metrics.	6
2.2	Heatmaps showing accuracy, bug detection capability, and code coverage across different difficulty levels and Research Cases (RCs) for the best closed-source model, GPT-4o, and the best open-source model, Llama-3.3-70B. The RCs under consideration are requirement (R), correct code (CC), incorrect code (IC), requirement and correct code (R-CC), and requirement and incorrect code (R-IC). Note that 0, 1, 1*, 2, 2*, 3, 3*, 4, 5, 6, 7 are progressively increasing difficulty levels (0 representing the lowest difficulty level) as determined by CodeChef.	10
3.1	Distribution of issues resolved by different number of agents in the SWT-Bench Lite (left) and SWT-Bench Verified (right) datasets.	21
3.2	Eight-set intersection upset plot for the selected agents from the SWT-Bench Verified dataset. Each colored dot indicates that the corresponding agent is included in that set. The number at the top of each black bar shows the count of agents in the intersection represented by the colored dots.	23

3.3	Venn diagrams showing the overlaps of issues resolved by the top three agents from the SWT-Bench Verified (left) and Lite (right) datasets. The number outside all circles indicates the number of issues that none of the three agents were able to resolve.	25
-----	---	----

List of Tables

2.1	Accuracy, bug detection capability, and code coverage using two closed-source and three open-source models on different research cases including requirements (R), correct code (CC), incorrect code (IC), requirement with correct code (R-CC), and requirement with incorrect code (R-IC). Note that the results of the research cases containing the three incorrect codes are combined into one by averaging their results for each problem. The best results are marked in bold. For each model and metric, we perform paired double-sided Mann Whitney U Test with a significance value (p-value) of 0.05 between each pair of research cases and find the effect size for the same. The pairs having absolute effect size ≥ 0.20 are marked with the same Greek letters. We noted that the pairs having noticeable effect size are subsets of the pairs that have a statistically significant difference. Do note that there are other pairs which have statistically significant differences but effect size < 0.20	16
3.1	Agents selected for this study and their corresponding success rates, as reported on the SWT-Bench Verified and Lite leaderboards [5].	20
3.2	Feature importance rankings (determined using random forest - mean decrease in impurity metric) for the selected agents from the SWT-Bench Verified and Lite datasets are presented. 1 denotes the most important feature, while 7 denotes the least important. Features ranked 2 through 6 lie in between, ordered by their relative importance.	27
3.3	The Pearson correlation coefficient for each agent–feature pair is presented. Positive values indicate a positive correlation, while negative values indicate a negative correlation. The largest positive and negative coefficients in each row are highlighted.	28

3.4	Feature importance, derived from logistic regression coefficients, is presented for the selected agents on the SWT-Bench Verified and Lite datasets. Positive coefficients indicate a positive correlation, while negative coefficients indicate a negative correlation. Statistical significance is denoted by ** (p-value ≤ 0.02) and * (p-value $\in (0.02, 0.05]$)	29
-----	---	----

Chapter 1

Introduction

In the software development life cycle, software testing accounts for over half of the development costs and resources [19], despite numerous advances in test automation [21]. Recent developments in Large Language Models (LLMs) and software development agents have significantly improved software engineering efficiency [30]. Yet, some key questions remain largely unanswered: what information should be provided to these models and agents? Do they perform better when given code directly, or do they benefit more from natural language descriptions? How important are requirements to their performance? Are they misled by incorrect or incomplete information?

Moreover, most studies attempt to use all available information without understanding or optimizing which aspects of it are most effective for LLMs and agents. Even commercial tools such as GitHub Copilot [6] and Qodo [31] focus primarily on generating tests directly from code, leaving a rich source of information (natural language requirements, task descriptions, and documentation) largely underutilized. As a result, these tools often generate tests that merely validate the code written by developers (which may itself contain bugs) rather than reasoning to detect and prevent errors. It is therefore critical to understand what LLMs and agents prioritize in the given prompt or context, and how these priorities influence their ability to generate effective tests.

1.1 Problem Statement

Test generation agents and LLMs are increasingly being used for test generation. This makes it essential to investigate:

- The extent to which different inputs can help LLMs and agents generate tests with high accuracy and coverage.
- A framework for evaluating LLMs based on different types of information provided in the prompts for a given task.
- The performance of current test generation agents on real-world bug-reproducing test generation.
- The features in bug reports that agents prioritize when generating tests.

This thesis explores these aspects to deepen our understanding of LLMs and test generation agents. Readers can derive insights into the effective use of agents and LLMs for test generation, as well as guidance to inform the design and architectural decisions for future agent and LLM development.

Thesis Statement: Understanding how different types of information provided to test generation agents and LLMs affect test generation.

1.2 Thesis Overview

This thesis is organized around two main studies - one examining LLMs and the other software development agents; each addressing key questions about how input and contextual information affect LLM and agent performance, followed by a discussion of their broader implications for practice and future research.

- **Chapter 2: What Inputs Drive Effective LLM-based Unit Test Generation?** This chapter examines the effectiveness of LLM-generated tests based on the information provided in prompts, using a competitive programming dataset (CodeChef) to evaluate five state-of-the-art LLMs.
- **Chapter 3: What Impacts the Bug-Reproducing Test Generation Capability of Agents?** This chapter goes beyond the previous study by analyzing a real-world software engineering dataset (SWT-Bench) to evaluate the influence of information provided in bug reports on bug-reproducing test generation, using eight state-of-the-art software development agents.

Chapter 2

What Inputs Drive Effective LLM-based Unit Test Generation?

Accepted in IEEE Software
DOI: [10.1109/MS.2025.3621625](https://doi.org/10.1109/MS.2025.3621625)

2.1 Introduction

Software testing consumes a staggering 50% of total development costs and over 40% of resources in the software development lifecycle [19]. Despite advances in test automation, testing still represents one of the most expensive aspects of software engineering, requiring substantial resources and effort to ensure comprehensive coverage and quality. Large Language Models (LLMs) have emerged as game-changers for test automation: trained on the corpus of software code and tests, LLMs can automatically generate high-quality executable test cases, dramatically enhancing developers' productivity in testing and finding bugs [30]. Yet a critical question remains unanswered: how does the context (code and natural language requirements) provided to LLMs impact the quality of the tests they generate?

Most research studies on LLM-based test generation (e.g., CodaMOSA [20] and CoverUp [26]) optimize for code coverage. Code coverage, while a great proxy, may not directly correlate with bug detection capability [16]. Additionally, these studies and commercial tools like GitHub Copilot [6] and Qodo [31] focus on generating tests directly from code, leaving

a rich source of information—natural language requirements from task descriptions and documentation—underutilized. While requirements could be used with these tools, we do not know if they will be useful. We address this gap with a comprehensive experiment that considers the correctness of code and usage of requirements.

We introduce a framework for evaluating the effectiveness of LLM-based test generation techniques with varying inputs, and apply it on two closed-source and three open-source LLMs using CodeChef dataset [23]. The focus of our study is to systematically investigate how the inclusion of different inputs—requirements, correct code, incorrect code, and their combinations—affect the quality of generated test cases across three key metrics: accuracy, bug detection capability, and code coverage.

A key finding of our study is that when the correctness of code is unknown, including requirements helps stabilize LLMs’ performance, i.e., generating tests of same or higher quality. If code is likely incorrect (e.g., when using LLMs to generate bug-revealing tests), relying solely on requirements is the most effective approach. We also observed smaller models struggle to infer specifications and input/output constraints from natural language requirements, leading to suboptimal results when generating test cases with requirements in the inputs.

A contemporary study on the influence of incorrect code on test generation [15] (on arXiv at the time of this submission) suggests that using correct code and including requirements lead to higher quality tests. Our work validates and extends the study to the most state-of-the-art LLMs and a challenging dataset with varying complexity levels and input/output constraints. Furthermore, our framework is the first to integrate test case reduction to standardize the test cases generated from LLMs with varying output verbosity. We also adopt human-written incorrect code submissions rather than LLM-synthesized ones to improve the validity of the bug detection capability measurement. We examine the impact of task difficulty on test case quality and use Mann-Whitney test with p-values and effect sizes as additional data-points to analyze results.

2.2 Experimental Design

This section outlines the key components of our study: dataset, LLM input construction, test generation and reduction processes, and evaluation metrics. This forms a comprehensive evaluation framework for LLM-based test generation, as illustrated in Figure 2.1.

2.2.1 Dataset

CodeChef (<https://www.codechef.com/>) is a widely recognized competitive programming platform offering coding problems spanning a diverse range of difficulty levels. Each problem comes with natural language requirements including a task description, input/output formats and constraints, and sometimes sample input/output pairs. User-submitted code is executed against a comprehensive test suite (private to CodeChef) to check for correctness. All user submissions, correct or incorrect, are publicly visible, making it ideal for our study.

We follow a similar procedure to a recent study on LLMs for test-driven development [23] to process the CodeChef dataset, using the same 1,100 problems—100 randomly sampled from 11 difficulty levels (focusing on Python).

For each problem, we extract the following information:

- requirement;
- one correct code submission (verified against CodeChef’s test suite);
- three distinct incorrect code submissions (that fail on some test cases in CodeChef’s test suite);
- metadata including the problem ID, difficulty score, and difficulty level.

To challenge LLMs in generating bug-revealing tests, we focus on partially correct submissions (labeled “wrong solution” by CodeChef) that pass syntax checks but contain logical flaws. We extract three such submissions per problem to ensure bug diversity. We found four problems with fewer than three incorrect-code submissions and replaced them with other problems of similar difficulty from CodeChef.

2.2.2 LLM Inputs Construction

To study how different inputs affect LLMs’ test generation quality, we design nine research cases (RCs), each of which constructs the LLM prompt with certain (combination of) information extracted from the CodeChef dataset for generating unit test cases:

- R: requirements only;
- CC: correct code only;

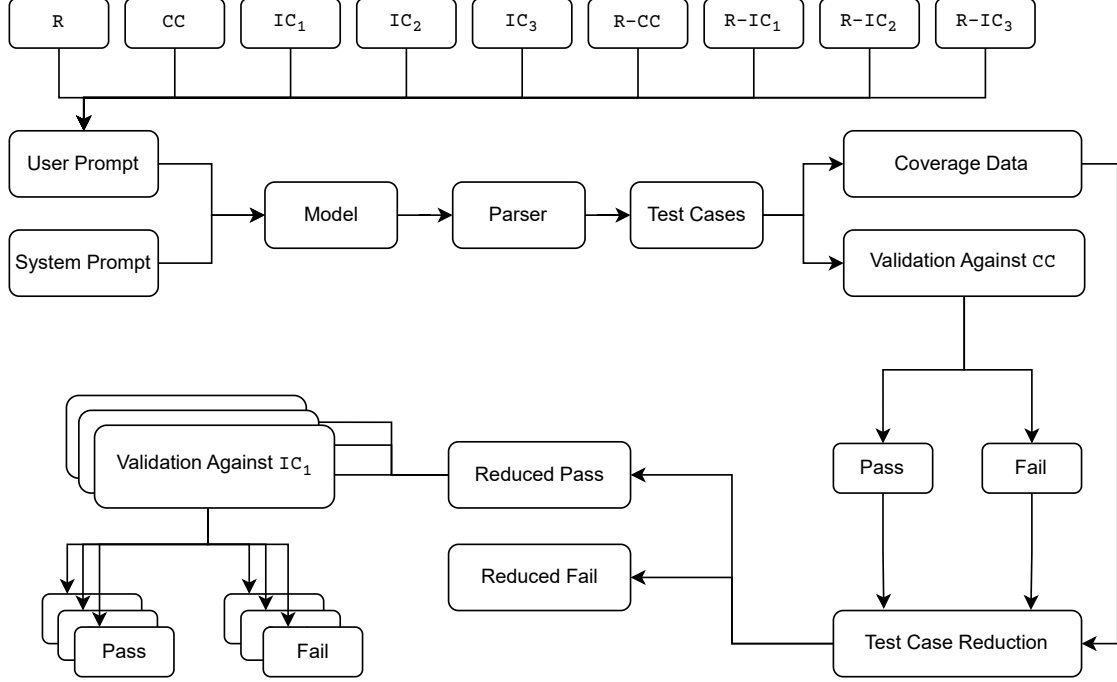


Figure 2.1: Workflow of our LLM-based test generation evaluation framework. Prompts containing different information form nine different research cases (requirements (R), correct code (CC), first incorrect code (IC₁), second incorrect code (IC₂), third incorrect code (IC₃), requirements and correct code (R-CC), requirements and first incorrect code (R-IC₁), requirements and second incorrect code (R-IC₂), and requirements and third incorrect code (R-IC₃)). These inputs are used to create the user prompt, which along with the system prompt is used by the LLM to generate test cases. These test cases are executed against the correct code to obtain pass/fail and code coverage data. This information is used for test case reduction and obtaining relevant evaluation metrics.

- IC₁, IC₂, IC₃: incorrect code only, using the first/second/third extracted incorrect code submission respectively;
- R-CC: requirements + correct code;
- R-IC₁, R-IC₂, R-IC₃: requirements + incorrect code, using the first/second/third extracted incorrect code submission respectively.

Determining the correctness of a given code is challenging. Thus, our experimental

setup generates a range of results to highlight areas for model improvement and optimal usage strategies. Details are provided in the Results section.

Following the best practices in prompting LLMs, we prepare a system prompt and a user prompt for each RC. The system prompt defines the model’s behavior and guidelines, clarifies the task (generating unit tests), and standardizes the output format for automated evaluation.

The system prompt is shown below, where the variable $\text{\textit{\$ \{provided_info_title\}}}$ specifies what information will be provided as inputs, e.g., “programming requirements and corresponding python code” for R-CC.

```
You are an expert Python Programmer and QA tester. Your job is to look at the  $\text{\textit{\$ \{provided\_info\_title\}}}$ 
provided by the user and output a comprehensive list of test cases for it.
Do not attempt to explain your output; simply provide correct test cases (including edge cases). The test
cases should be in the format of:
    assert main(input1) == "expected_output1"
    assert main(input2) == "expected_output2"
Note that the function name in the assert statement will always be main and the input to main will be a
string that will be used as a standard input stream. There should be NO comments.
```

The user prompt is used to provide information based on the specific RC, reflecting the user’s request and context during inference. For example, the user prompt for R-CC simply contains the requirements and correct code, as shown below:

```
The requirements are:
 $\text{\textit{\$ \{requirement\}}}$ 

and the corresponding Python code is:
 $\text{\textit{\$ \{correct\_code\}}}$ 
```

2.2.3 Test Generation and Reduction

Both prompts are passed to the LLM for inference using zero-shot prompting. The LLMs are hosted on Nvidia RTX 6000 Ada GPUs using Ollama (which adjusts the special tokens configurations specific to the LLM). We perform greedy decoding by setting the temperature to 0, which leads to high-quality outputs and also ensures deterministic and reproducible evaluation. The maximum token limit is set to 5,000 ($\sim 3,750$ words), which is sufficient for the LLMs to generate comprehensive test cases.

Our prompts guide the LLM to generate test cases in a specific format. We create a parser that extracts these tests using pattern matching and auto-corrects minor formatting

errors (e.g., removing comments, text, code block delimiters). If the model output deviates significantly (e.g., natural language test plans), it’s discarded as unusable.

After generating test cases, we validate them against the correct code with a pass/fail outcome and use Python’s coverage.py module to measure code coverage.

In our preliminary experiments, we found that LLMs may generate similar or redundant tests that don’t improve code coverage, potentially skewing results. To address this, we use a greedy algorithm [33] for coverage-based test case reduction, removing tests that don’t contribute to coverage increase. Note that we first group tests by outcome (pass/fail), and then perform reduction separately to each group (reduced pass/fail). This ensures a fair comparison of tests across LLMs at different verbosity levels. For example, if an LLM generates 10 tests, 9 passing but covering the same code-paths and 1 failing, without reduction, this would suggest 90% accuracy. After reduction, we keep just two tests, reflecting a more realistic 50% accuracy.

2.2.4 Evaluation Metrics

We evaluate quality of the generated test cases, after test case reduction, using three metrics.

Accuracy measures how well the LLM generates tests that pass against correct code, reflecting its ability to understand code and/or requirements and create syntactically and semantically valid tests. For a given RC, the accuracy (A_{RC}) is calculated as the average percentage of reduced correct test cases over the dataset:

$$A_{RC} = \frac{\sum_{i=1}^N \frac{c_{RC,i}}{c_{RC,i} + f_{RC,i}}}{N} * 100 \quad (2.1)$$

where $c_{RC,i}$, $f_{RC,i}$ is the number of reduced correct/incorrect test cases for problem i under the RC, and N is the dataset size (1,100).

For RCs involving multiple incorrect code versions (IC and R-IC), we average results across the three versions (i.e., $A_{IC} = (A_{IC_1} + A_{IC_2} + A_{IC_3})/3$ and $A_{R-IC} = (A_{R-IC_1} + A_{R-IC_2} + A_{R-IC_3})/3$) since there is no inherent ordering between them.

Bug Detection Capability evaluates how effectively the generated test cases can identify bugs in incorrect code. A high-quality test suite should pass on correct code while failing on buggy code. The bug detection capability for a given RC (B_{RC}) is calculated as:

$$B_{RC} = \frac{\sum_{i=1}^N \sum_{j=1}^3 \text{sgn}(\frac{g_{RC,i,IC_j}}{c_{RC,i}})}{3 * N} * 100 \quad (2.2)$$

where sgn is the sign function (returning 1 when input is positive, 0 otherwise), g_{RC,i,IC_j} represents the number of reduced correct test cases that fail when executed against incorrect implementation IC_j for problem i . The usage of sgn implies that to consider a bug being detected by a test suite, it is sufficient for at least one test case to fail on the incorrect code. Similar to accuracy, we average the results for RCs involving multiple incorrect code versions.

Code Coverage measures how thoroughly the generated tests exercise the implementation code. We measure line-level code coverage using Python’s `coverage.py` module, which is the percentage of lines covered by the reduced correct test cases divided by the total number of lines in the correct code. The code coverage for a given RC (C_{RC}) is calculated as the average line coverage on each problem in the dataset, similar to how accuracy is calculated.

2.3 Results

Our primary research question explores how different inputs affect the quality of LLM-generated test cases across accuracy, bug detection, and code coverage. We use a paired two-sided Mann-Whitney U test to compare different input sets on each dimension (where each input is an array representing one of R, CC, IC, R-CC, R-IC and contains the corresponding metric values computed for each problem). We report the pairs with noticeable absolute effect size (≥ 0.2) that are also statistically significantly different (p-value < 0.05).

2.3.1 Models

We experimented with five state-of-the-art LLMs with varying sizes and open-source status:

- **Closed-sourced:** GPT-4o (2024-08-06) and GPT-4o-mini (2024-07-18);
- **Open-sourced:** Llama-3.3-70B, Codestral-22B, and Llama-3.1-8B (all instruction-tuned).

2.3.2 Test Case Quality

Table 2.1 presents the main results of our study, showing the quality of the test cases generated by five state-of-the-art LLMs across RCs.

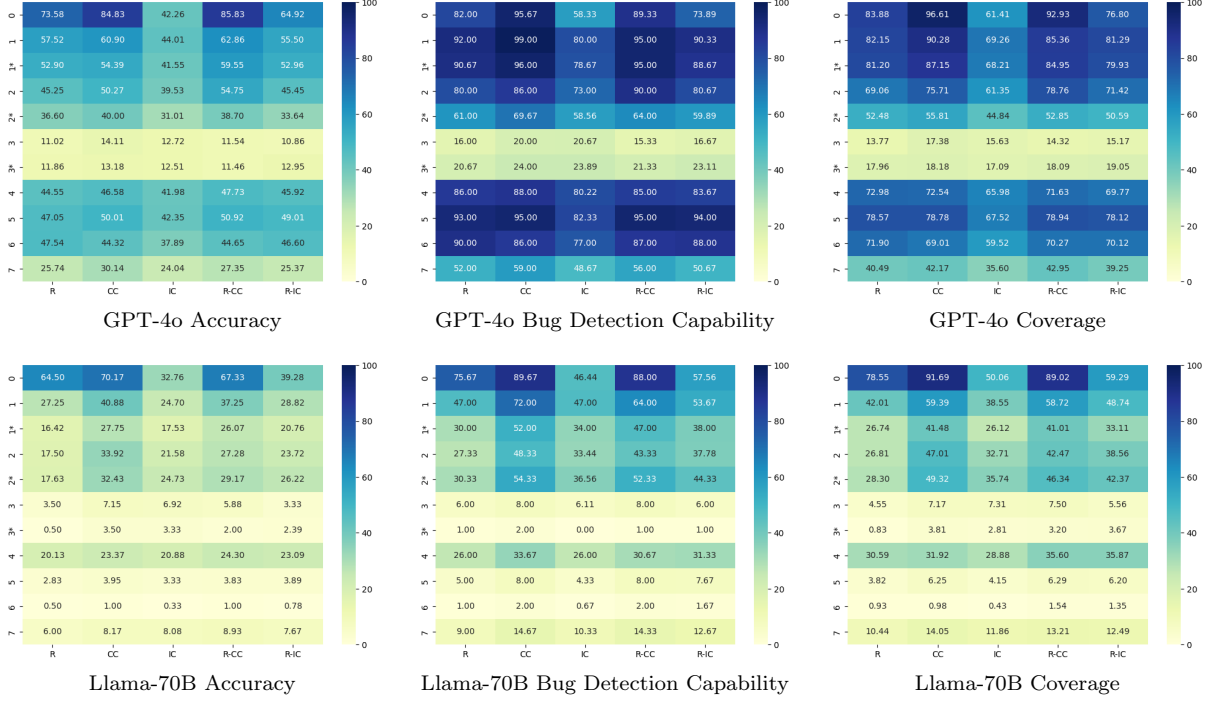


Figure 2.2: Heatmaps showing accuracy, bug detection capability, and code coverage across different difficulty levels and Research Cases (RCs) for the best closed-source model, GPT-4o, and the best open-source model, Llama-3.3-70B. The RCs under consideration are requirement (R), correct code (CC), incorrect code (IC), requirement and correct code (R-CC), and requirement and incorrect code (R-IC). Note that 0, 1, 1*, 2, 2*, 3, 3*, 4, 5, 6, 7 are progressively increasing difficulty levels (0 representing the lowest difficulty level) as determined by CodeChef.

Closed-source Models

When using closed-source LLMs (GPT-4o and GPT-4o-mini), including both requirements and correct code (R-CC) in the prompt results in the highest accuracy (47.24% and 41.94%, respectively). Providing only correct code (CC) or only requirements (R) produces the next best results. Notably, including only incorrect code in the prompt (IC) results in the lowest accuracy (35.34% and 31.64%, respectively). However, augmenting incorrect code with requirements (R-IC) significantly increases the accuracy by approximately 19.38%. This trend extends to bug detection capability and coverage, where, on average, CC yields the highest scores (e.g., 74.39% bug detection capability and 63.97% code coverage for GPT-4o), while including only incorrect code (IC) yields the lowest scores (e.g., 61.94% bug detection capability and 51.49% code coverage for GPT-4o).

We observe that generating tests using CC is statistically significantly better than using IC alone (with noticeable effect size) indicating that when given only code, the models assume that it is correct and do not focus on the actual natural language text in the code—also seen from IC having the lowest bug detection capability (e.g., the variable name `add_a.b` in the expression `add_a.b = a*b` likely makes no difference to the model even though we notice a probable logical error at a glance).

When given only incorrect code, models assume it is correct and generate tests to validate it. In closed-source models, combining requirements with incorrect code slightly lowers performance compared to using requirements alone, as the incorrect code introduces contradictions and misleads the model.

We observe that the range of accuracy for GPT-4o when providing only code (correct or incorrect) is [35.34%, 47.07%], but augmenting code with requirements improves the range to [42.25%, 47.24%]. The same enhancement pattern appears for GPT-4o-mini and repeats for bug detection capability and code coverage metrics as well, indicating that adding requirements to code helps improve the overall quality of the test suite. Furthermore, the results for generating tests using only requirements (R) fall within the ranges of R-IC and R-CC.

Without knowing whether the given code is correct, generating test cases using only requirements or augmenting code with requirements are the most effective approaches.

Test cases generated using only IC are statistically significantly different from those using R-IC. Since the accuracy of R-IC surpasses that of IC, this further confirms that requirements help generate higher quality test cases. A similar trend is observed for bug detection capability and code coverage.

Augmenting incorrect code with requirements helps detect more bugs and increases code coverage.

Open-source Models

Test generation using only correct code yields the best accuracy (22.93%, 18.01%, and 11.94% for Llama-3.3-70B, Codestral-22B, and Llama-3.1-8B, respectively). For Llama-3.3-70B, adding requirements to both correct and incorrect code results in the next two best accuracies (21.19% and 16.36%). Similar to closed-source models, generating test cases using only incorrect code yields the lowest accuracy. We observe that passing requirements to Codestral-22B and Llama-3.1-8B negatively impacts accuracy.

For bug detection capability and coverage, Llama-3.3-70B and Codestral-22B do the best using CC and R-CC, and the worst when using IC as seen in Table 2.1. Llama-3.1-8B obtains the highest bug detection and coverage using CC, lowest bug detection using R-IC and the lowest coverage using R.

Therefore, for Llama-3.3-70B, incorporating requirements into IC improves test quality, though not significantly. Adding requirements to CC does not enhance the test quality and may even degrade it.

For Codestral-22B and Llama-3.1-8B, adding requirements to code perplexes the model and results in similar or lower accuracy. Using only requirements performs worse than using IC.

Smaller models ($\leq 22B$ parameters) struggle to infer specifications and input/output constraints from natural language, compared to using only code.

2.3.3 Difficulty Level

Figure 2.2 visualizes the metrics across various problem difficulty levels in our dataset. A general decrease in value across all the metrics is observed. This indicates that as the complexity of the problems grows, the quality of the generated test cases decreases. However, we also observe some irregularities in this overall trend, likely due to differences in pretraining data and optimization strategies for each LLM.

Using Mann-Whitney test to analyze differences between the easiest problems (difficulty levels 0-1) and the most difficult problems (levels 6-7) reveals notable patterns. For the easiest problems, adding requirements to code (both, correct and incorrect) or just using requirements, compared to incorrect code resulted in statistically significant differences with substantial effect size (≥ 0.30) between them. This indicates that incorporating requirements into code or simply using requirements substantially improves test quality. In contrast, for the most difficult problems, although the same comparisons were statistically significant, the effect sizes were small. This suggests that the practical impact of prompt information diminishes as problem difficulty increases. Similar trends were observed across all evaluation metrics for both GPT-4o and Llama 70B.

When the task is difficult, using more inputs does not help much likely due to model's limitation.

2.3.4 Impact of Model Size

Larger models benefit from both code and requirements, improving test case quality, while smaller models struggle with natural language and perform better using only code.

As model size decreases, the quality of the generated test cases also decreases.

2.4 Implications

1. Each LLM exhibits distinct strengths and weaknesses influenced by its size, architecture, and pretraining dataset. Our results show that test quality generally decreases

with model size. Developers must make informed decisions when selecting an LLM for test generation, considering task complexity and input context. They may need to adjust prompting strategies and provide complete context to optimize performance.

2. Requirements play a critical role in generating high-quality test cases, especially since developers cannot always be certain of the correctness of the code under test. Larger models effectively synthesize information from both code and requirements to produce higher-quality test cases. In contrast, smaller models may struggle, with their performance heavily dependent on the complexity of the provided requirements.
3. Depending on the selected model, optimizing for both test coverage and bug detection may not be feasible. Developers often need to prioritize one and subsequently address the other based on the codebase and development stage, interpreting both metrics cautiously as indicators of test quality. If both metrics are desired simultaneously, top-performing models like GPT-4o are recommended.
4. In real-world software development, the effectiveness of LLM-based test generation depends on the availability of high-quality requirements and correct code. While correctness of added code can't be guaranteed regardless of codebase maturity, having clear requirements helps. Mature codebases offer the added benefit of using existing, likely correct code as context, and may enable requirement generation via LLMs—an interesting direction for future research.

2.5 Conclusion

We present an empirical study that investigates how different inputs affect the quality of generated test cases across five state-of-the-art LLMs. Our observations reveal that well-defined requirements significantly enhance test quality. However, overly rigid input/output formatting requirements can degrade test quality for smaller models rather than improving it. Overall, requirements—essentially natural language descriptions of desired outcomes—are a powerful and widely applicable way to unlock greater value from LLMs.

Developers can boost their productivity with LLMs for test generation by choosing the right model for their specific needs. However, instead of blindly trusting LLMs' responses, developers should critically analyze outputs for potential errors, i.e., positioning LLMs as valuable assistants while maintaining essential human oversight.

We acknowledge that competitive programming, as represented by the CodeChef dataset, may differ from real-world software development. The problems in such datasets may lack

the complexity, diversity, and context-driven aspects that characterize real-world software development tasks.

Additionally, our work points to several promising research directions: (1) An interesting idea would be the use of a feedback loop, where the model uses some information it generates to iteratively enhance its responses. However, this lies outside the scope of our current work since real-world software lacks the ground truth regarding the correctness of underlying code or the generated test cases (another research problem), making the implementation of an effective feedback mechanism challenging without additional signals. Our framework uses a dataset that has information about correctness of the code, a luxury that is often not available in the real-world. A promising avenue, however, could involve leveraging other models to compare generated outputs against external signals like meeting notes, documentation, or other development artifacts, to help the model refine its responses more effectively. (2) exploring how the prompt structure (like code/requirements length and complexity) affect test case quality (3) analyzing whether reasoning-enabled LLMs outperform non-reasoning LLMs; (4) model finetuning to generate code-correctness agnostic bug-revealing tests; (5) improving smaller models using model distillation. Future work can reuse our evaluation framework to assess the effectiveness of new LLM-based test generation techniques.

Model	R	CC	IC	R-CC	R-IC
Accuracy					
GPT-4o	43.54	^{α} 47.07	^{$\alpha\beta$} 35.34	^{β} 47.24	42.25
GPT-4o-mini	38.23	^{γ} 41.87	^{γ} 31.64	41.94	37.70
Llama-3.3-70B	16.07	22.93	14.93	21.19	16.36
Codestral-22B	11.88	18.01	14.92	15.18	13.09
Llama-3.1-8B	^{δ} 2.86	11.94	^{$\delta\epsilon\zeta$} 10.56	^{ϵ} 3.37	^{ζ} 2.77
Bug Detection Capability					
GPT-4o	69.39	74.39	61.94	72.09	68.14
GPT-4o-mini	67.36	72.76	60.29	70.55	65.61
Llama-3.3-70B	23.48	34.97	22.26	32.61	26.52
Codestral-22B	19.27	26.67	20.95	24.42	20.97
Llama-3.1-8B	2.45	6.85	3.57	3.03	1.97
Coverage					
GPT-4o	60.4	^{η} 63.97	^{η} 51.49	62.82	59.23
GPT-4o-mini	56.79	59.18	48.39	58.88	54.58
Llama-3.3-70B	23.05	32.10	21.69	31.35	26.11
Codestral-22B	18.60	25.09	19.84	23.90	20.89
Llama-3.1-8B	^{θ} 3.14	11.86	^{$\theta\iota\kappa$} 8.95	^{ι} 3.96	^{κ} 3.28

Table 2.1: Accuracy, bug detection capability, and code coverage using two closed-source and three open-source models on different research cases including requirements (R), correct code (CC), incorrect code (IC), requirement with correct code (R-CC), and requirement with incorrect code (R-IC). Note that the results of the research cases containing the three incorrect codes are combined into one by averaging their results for each problem. The best results are marked in bold. For each model and metric, we perform paired double-sided Mann Whitney U Test with a significance value (p-value) of 0.05 between each pair of research cases and find the effect size for the same. The pairs having absolute effect size ≥ 0.20 are marked with the same Greek letters. We noted that the pairs having noticeable effect size are subsets of the pairs that have a statistically significant difference. Do note that there are other pairs which have statistically significant differences but effect size < 0.20 .

Chapter 3

What Impacts the Bug-Reproducing Test Generation Capability of Agents?

3.1 Introduction

Bug reports (also known as issues) are used to describe bugs, feature requests, or other suggestions for improving a software repository. They allow developers to be informed of problems encountered while using a software [34]. Manually resolving issues is time-consuming, highlighting the need to automate and modernize this process. Artificial Intelligence (AI)-based software engineering is rapidly evolving, with the next step being software development agents capable of performing development tasks [12]. Recently, there has been significant progress in software development agents for tasks such as code generation [13], code repair [27], and test case generation [21].

Test generation agents are increasingly being used to generate bug-reproducing tests. With the growing variety of available agents, each exhibiting different levels of performance, it is essential to understand which aspects of bug reports these agents leverage and which features contribute to their effectiveness. Insights into these factors can guide the design of more capable agents and provide developers with actionable guidance on how to write higher-quality bug reports, thereby advancing the broader field of AI-driven software engineering.

In this paper, we leverage the SWT-Bench benchmark [24] to analyze state-of-the-art

(SOTA) agents from the SWT-Bench Verified and Lite leaderboards [5], including LogicStar AI [4], OpenHands [3], e-otter++ [11], otter++ [10], Amazon Q [1], and AssertFlip [18], to evaluate their effectiveness and issue coverage in bug-reproducing test generation. We also manually annotate the SWT-Bench Lite and Verified datasets to identify key features present in bug reports (as described in the Labeling subsection of Study Design). Using these annotations, we perform statistical analyses to identify which features most significantly influence an agent’s ability to generate bug-reproducing tests. We also analyze the differences in characteristics between issues that the top-ranked agent on the leaderboard resolves but other agents fail to resolve, and vice versa. Additionally, we perform qualitative analysis of issues that all agents fail to resolve, except for those ranked at the bottom of the leaderboard.

The main contributions of this paper are as follows:

- We evaluate the effectiveness of state-of-the-art test generation agents by analyzing how many issues are resolved by each agent and the extent of overlap among the issues they resolve.
- We identify key features in the issues from SWT-Bench Lite and Verified datasets and manually label all instances based on the presence or absence of these features.
- We perform statistical analyses on the labeled data to determine the relative influence of different features on agents’ ability to generate bug-reproducing test cases.
- To support reproducibility and further research, we make our datasets and code publicly available.

3.2 Study Design

To achieve our goal of identifying the characteristics of bug reports that influence the bug-reproduction capability of software development agents, we use a real-world software engineering benchmark and manually annotate it based on specific features. We then analyze the data to answer our research questions. This section provides details on the selected benchmark and agents, the labeling methodology, and the research questions guiding our study.

3.2.1 Benchmark

We use the SWT-Bench Verified and Lite datasets, which are subsets of the full SWT-Bench dataset [24]. SWT-Bench is an evaluation framework created to measure how effectively software development agents and Large Language Models (LLMs) can generate unit tests that reproduce real-world GitHub issues. It is derived from the SWE-bench dataset [17], which evaluates LLMs and software agents on their ability to autonomously resolve real-world software engineering issues by generating patches for GitHub issues in popular Python repositories.

All software development agents on the leaderboards have been evaluated using the Verified and Lite versions of SWT-Bench, which is why we focus on these subsets. SWT-Bench Verified [8] contains 433 human-verified solvable instances, a subset of the 500 instances in SWE-Bench Verified [2], which were meticulously validated by human developers. SWT-Bench Lite [7] contains 276 instances, a subset of the 300 instances in SWE-Bench Lite [9].

3.2.2 Agents

We select all agents listed on the SWT-Bench Verified leaderboard (as of February 2026), excluding those without publicly available execution logs (OpenHands GPT-5) and models that do not qualify as agents (Libro and Zero-Shot Plus). From the SWT-Bench Lite leaderboard, we select the top three agents (as of February 2026). Table 3.1 summarizes the selected agents along with their reported issue resolution rates.

For each agent, we parse the execution logs to extract the **resolved** metric for each instance, which we use in our analysis. According to the SWT-Bench paper [24], the **resolved** (or success) metric is a binary value: it is true if at least one of the generated tests transitions from failing (on the codebase where the bug is present) to passing (on the codebase where the patch fixing the issue is applied) and none fail after applying the patch; otherwise it is false. The success rate reported by SWT-Bench is calculated as the average number of resolved instances across the dataset. As this is the primary metric introduced in the SWT-Bench paper, it is well-suited for evaluating the performance of software development agents in our study.

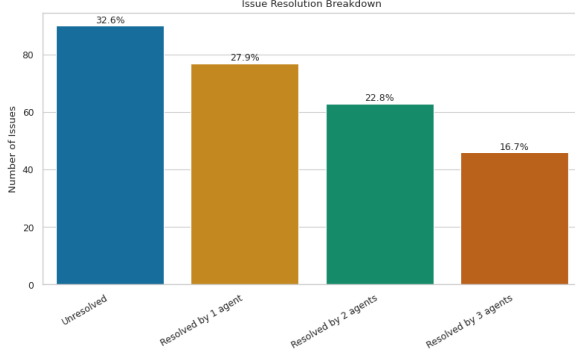
Table 3.1: Agents selected for this study and their corresponding success rates, as reported on the SWT-Bench Verified and Lite leaderboards [5].

Agent	Success Rate
SWT-Bench Verified	
LogicStar AI (L*Agent v1)	84.0%
OpenHands (GPT-5-mini)	62.4%
e-Otter++ (Claude Sonnet 3.7)	62.1%
Amazon Q Developer Agent (v20250405-dev)	51.0%
AssertFlip (GPT-4o)	45.5%
Otter++ (GPT-4o)	37.4%
Otter (GPT-4o)	31.6%
OpenHands (Claude Sonnet 3.5)	27.7%
SWT-Bench Lite	
e-Otter++ (Claude Sonnet 3.7)	52.5%
Amazon Q Developer Agent (v20250405-dev)	39.9%
AssertFlip (GPT-4o)	38.0%

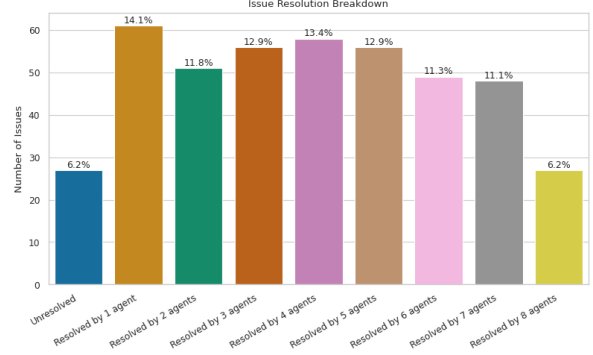
3.2.3 Labeling

To evaluate which features in bug reports influence the performance of software development agents, we extracted the instance IDs and corresponding issue descriptions (problem statements) from the SWT-Bench Verified and Lite datasets. We then manually annotated each instance according to the presence of specific features in the issue description. Each instance was assigned a binary label (Yes/No) for the following features:

- **ic_nl**: Indicates whether the issue provides a natural language description of the incorrect behavior.
- **ic_eg**: Indicates whether the issue includes examples or stack traces to illustrate the incorrect behavior.
- **c_nl**: Indicates whether the issue provides a natural language description of the correct behavior.
- **c_eg**: Indicates whether the issue includes examples (expected output) to illustrate the correct behavior.



(a) SWT-Bench Lite



(b) SWT-Bench Verified

Figure 3.1: Distribution of issues resolved by different number of agents in the SWT-Bench Lite (left) and SWT-Bench Verified (right) datasets.

- **s_nl**: Indicates whether the issue describes the proposed solution or specifies what changes are required to resolve the issue in natural language.
- **s_code**: Indicates whether the issue provides code that implements or directly suggests a solution.
- **s_loc**: Indicates whether the issue includes a code snippet containing the bug or explicitly localizes the bug by identifying the specific source code files or methods responsible for the reported problem. Although stack traces and examples may sometimes provide localization information, they do not always clearly identify the bug’s location. Therefore, localization information inferred solely from stack traces or examples is not considered when labeling this feature.

3.2.4 Research Questions

We use the labeled SWT-Bench Verified and Lite datasets to answer the Research Questions (RQs) mentioned below.

RQ1

How effective are current software development agents when generating bug-reproducing tests?

The SWT-Bench Verified and Lite datasets present tasks in which agents must generate bug-reproducing tests for issues in large and complex codebases. While the leaderboard reports metrics such as success rate and line coverage for each agent, we want to go beyond these basic measures. We analyze the number of issues resolved by each agent and examine overlaps among the issues resolved by different agents. This analysis provides insight into each agent’s effectiveness and helps assess whether the top-performing models are sufficient for the task or whether combining multiple agents could leverage complementary strengths to achieve better overall results.

RQ2

Which features in bug reports play the most influential role in helping software development agents generate bug-reproducing tests?

Each agent differs in its underlying LLM and architecture, resulting in variations in performance. Consequently, agents analyze issues differently when generating bug-reproducing tests and may prioritize different features. To understand these differences, we perform statistical analysis of the labeled SWT-Bench Verified and Lite datasets to identify the relative importance of features in helping agents generate bug-reproducing test cases.

RQ3

How do bug report features differ between issues uniquely resolved by the top-performing agent and those resolved by other agents but missed by the top-performing agent?

We use our findings from RQ1 to select issues uniquely resolved by the top-performing agent among the selected agents and analyze patterns in the presence or absence of features using the labeled SWT-Bench Verified dataset. We perform a similar analysis for issues that the top-performing agent could not resolve but were successfully resolved by at least one other agent. This approach helps us identify which features in a bug report the top agent prioritizes. Additionally, we conduct a qualitative analysis of issues resolved by agents at the bottom of the leaderboard but not by other agents, highlighting how different agents can exhibit complementary strengths.

3.3 Analysis and Results

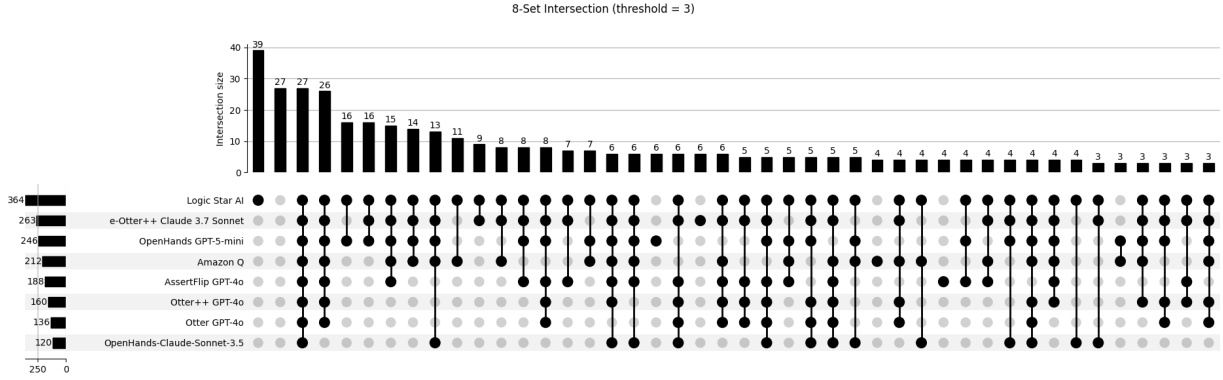


Figure 3.2: Eight-set intersection upset plot for the selected agents from the SWT-Bench Verified dataset. Each colored dot indicates that the corresponding agent is included in that set. The number at the top of each black bar shows the count of agents in the intersection represented by the colored dots.

3.3.1 RQ1: How effective are current software development agents when generating bug-reproducing tests?

We analyze the performance of agents using the number of issues resolved by each agent (using the resolved metric as discussed in the Study Design Section) and examine the overlaps among the issues resolved by different agents using the SWT-Bench Verified and Lite datasets.

Issue Resolution Breakdown

Figure 3.1 shows the breakdown of issue resolutions by agents on the SWT-Bench Lite and verified dataset. We observe that out of the 276 issues in SWT-Bench Lite, 67.39% (186) of them were resolved by at least one agent and 32.61% (90) are left unresolved by the selected agents. This indicates that a substantial portion of issues could not be addressed by the top three agents (having overall success rate of 38% or greater). Additionally, 27.9% of the issues were resolved by one agent only, and a relatively small proportion (16.7%) were solved by all three agents.

On the other hand, in the SWT-Bench Verified dataset, out of the 433 issues, 406 were resolved by at least one agent and 27 remain unresolved among the selected agents (394 resolved and 39 unresolved when considering only the top three agents). Only 14.1% of the issues were resolved by a single agent, and a relatively small proportion (6.2%) were

resolved by all eight agents. We observe a similar proportion (about 11%) of issues resolved by two to seven agents. This suggests that while some issues can be resolved by all agents, many require unique capabilities from different agents.

The bug-reproducing test generation capability of current software development agents is impressive; however, no single agent consistently dominates, highlighting the need for further improvements - particularly for issues that remain unresolved by all agents or those that very few agents can resolve.

Unique issue coverage

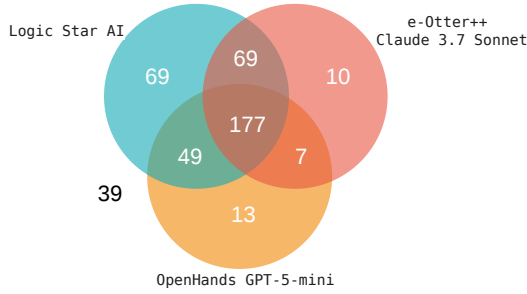
Figure 3.2 presents an UpSet plot illustrating the eight-set intersections for the selected agents from the SWT-Bench Verified leaderboard, sorted by set cardinality (with a threshold of three). As shown, Logic Star resolves 39 issues (9%) that no other agent is able to resolve. In addition, 27 issues are resolved by all agents, while another 27 remain unresolved by any agent. The top three to five agents exhibit overlapping resolutions ranging from six to 26 shared issues. Moreover, e-Otter++, and OpenHands GPT-5-mini resolve six issues that no other agent can resolve. Amazon Q and AssertFlip each resolve four such issues. OpenHands Claude Sonnet 3.5 (the relatively worst-performing agent) resolves two issues that no other agent resolves, while Otter and Otter++ collectively resolve one issue that no other agent resolves. These results highlight the unique perspectives and strengths of different agents, regardless of their overall performance.

Furthermore, among the top three agents on SWT-Bench Verified and Lite (see Figures 3.3a and 3.3b), each agent resolves a substantial number of unique issues (ranging from 10 to 69) that the others fail to address.

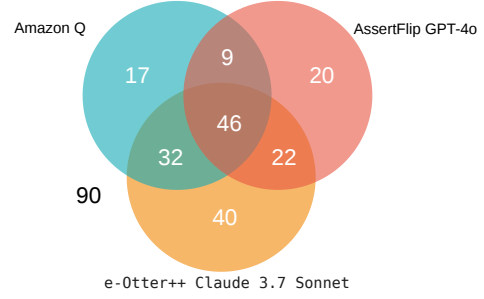
We also observe that Logic Star AI (the best performing agent on SWT-Bench Verified) fails to resolve 42 issues that are successfully resolved by at least one other agent. We examine this further in RQ3.

These findings highlight the diversity in agent capabilities. Although the top-performing agents achieve strong overall metrics, each demonstrates strengths on distinct subsets of issues that are not fully captured by aggregate performance measures.

Each agent has its unique strengths that enables it to resolve issues that even the best state-of-the-art agent fails to resolve. This diversity in the strengths of agents shows the complementary roles the agents play in resolving issues.



(a) Venn Diagram for top three agents of SWT-Bench Verified



(b) Venn Diagram for top three agents of SWT-Bench Lite

Figure 3.3: Venn diagrams showing the overlaps of issues resolved by the top three agents from the SWT-Bench Verified (left) and Lite (right) datasets. The number outside all circles indicates the number of issues that none of the three agents were able to resolve.

3.3.2 RQ2: Which features in bug reports play the most influential role in helping software development agents generate bug-reproducing tests?

We perform statistical analysis on the labeled SWT-Bench Verified and Lite datasets to assess the relative importance of various features to agents’ success in generating bug-reproducing test cases. To this end, we used random forest feature importance (mean decrease in impurity, MDI), logistic regression, and Pearson’s correlation coefficient.

We trained a random forest and logistic regression model on the entire dataset using the seven features, described in the Labeling subsection of the Study Design section, as independent variables and the **resolved** metric for each agent as the dependent variable (as described in the Agents subsection of the Study Design section). Binary labels (Yes/No and True/False) were encoded as 0 and 1, respectively. We set the number of estimators in random forest to 500 and observed similar results for values ranging from 100 to 1000, with all other hyper-parameters left at their default settings. We also used the Pearson correlation coefficient to examine the direction of the relationships between the features and agent success. The resulting relative feature importance and correlation is reported in Tables 3.2, 3.3, and 3.4. Note that the random forest algorithm is run with 200 different `random.state` values, and the reported feature importance is the mean across these runs.

SWT-Bench Verified

Based on Tables 3.2, 3.3, and 3.4, the overall trend indicates that solution in natural language (`s_nl`) is the most influential (with positive correlation) for bug-reproducing test case generation, impacting five of the eight agents when considering top three important features (three agents if only considering most important feature). This is followed by bug localization information (`s_loc`) for four agents (two if considering the top-most feature only). Correct behavior in natural language is also one of the most influential features that appears seven times considering top three (or two times considering the top-most feature). However for two cases (Logic Star AI and Amazon Q) out of the seven, it is negatively correlated.

SWT-Bench Lite

We observe that bug localization information (`s_loc`) is the most influential feature, exhibiting a positive correlation and appearing among the top three features for all agents. This is followed by the solution described in natural language (`s_nl`). Correct behavior described in natural language is also among the most influential features and appears for all agents; however, it is negatively correlated for one agent.

Additional Discussion

From a statistical perspective, `ic_nl` is expected to have the lowest importance on SWT-Bench Lite and Verified, as it appears in nearly all instances and thus offers limited discriminative value. In practice, however, it remains an essential feature, since neither an agent nor a human can fully understand the bug or the required fix without a description of the issue.

We do not observe a consistently similar ranking of features across all agents. This is expected, as each agent has a distinct architecture and underlying LLM(s), leading to different prioritization of features in bug reports. For example, AssertFlip employs a “pass-then-invert” architecture, that generates tests that pass on the buggy code and then inverts it. Since `ic_nl` represents the minimal baseline information in a bug report, it can be used to generate passing tests for buggy code. Additionally, it may leverage `s_loc` to identify where the bugs are located and determine where the tests should be placed.

Otter’s architecture (Otter++/Otter) centers around generating code patches and tests that pass on “correct code” (i.e., codebase where the bug-fixing patch has been applied).

This aligns with our findings, where `c_nl` and `c_eg` are the most important features with a positive correlation, as they support the generation of tests that pass on correct code.

Overall, we observe that solution in natural-language, localization and correct behavior in natural language play the most influential roles in generating bug-reproducing tests. Specifically, each agent may be influenced by different features, often due to variations in their architectures and underlying LLMs.

Table 3.2: Feature importance rankings (determined using random forest - mean decrease in impurity metric) for the selected agents from the SWT-Bench Verified and Lite datasets are presented. 1 denotes the most important feature, while 7 denotes the least important. Features ranked 2 through 6 lie in between, ordered by their relative importance.

Agent	ic_nl	ic_eg	c_nl	c_eg	s_nl	s_code	s_loc
SWT-BENCH VERIFIED							
Logic Star AI	7	3	2	6	4	5	1
OpenHands GPT-5-mini	7	2	3	4	1	6	5
e-Otter++ Claude 3.7	7	5	4	1	3	6	2
Amazon Q Developer	7	5	3	4	1	2	6
AssertFlip GPT-4o	7	4	2	5	3	6	1
Otter++ GPT-4o	7	5	1	2	4	6	3
Otter GPT-4o	7	3	1	2	4	6	5
OpenHands Claude 3.5	7	2	3	5	1	6	4
SWT-BENCH LITE							
e-Otter++ Claude 3.7	7	3	2	5	4	6	1
Amazon Q Developer	7	4	2	6	1	5	3
AssertFlip GPT-4o	7	4	1	6	3	5	2

Table 3.3: The Pearson correlation coefficient for each agent–feature pair is presented. Positive values indicate a positive correlation, while negative values indicate a negative correlation. The largest positive and negative coefficients in each row are highlighted.

Agent	ic_nl	ic_eg	c_nl	c_eg	s_nl	s_code	s_loc
SWT-BENCH VERIFIED							
Logic Star AI	0.002	0.077	-0.022	-0.018	-0.014	-0.025	0.105
OpenHands GPT-5-mini	-0.063	-0.063	0.003	-0.056	0.021	-0.000	0.054
e-Otter++ Claude 3.7	0.026	-0.010	0.008	-0.111	0.093	0.094	0.107
Amazon Q Developer	-0.003	0.090	-0.027	0.012	0.065	0.015	0.074
AssertFlip GPT-4o	-0.056	0.009	0.004	0.024	0.020	0.016	0.104
Otter++ GPT-4o	0.009	0.043	0.020	0.023	0.026	0.006	0.091
Otter GPT-4o	-0.005	0.031	0.043	0.006	0.010	0.007	0.074
OpenHands Claude 3.5	0.029	0.038	0.045	-0.003	0.009	-0.042	0.076
SWT-BENCH LITE							
e-Otter++ Claude 3.7	0.023	0.025	0.032	-0.111	0.121	0.080	0.036
Amazon Q Developer	0.059	0.065	0.035	-0.013	0.150	0.100	0.099
AssertFlip GPT-4o	0.050	0.038	-0.043	-0.075	0.027	0.058	0.078

3.3.3 RQ3: How do bug report features differ between issues uniquely resolved by the top-performing agent and those resolved by other agents but missed by the top-performing agent?

We observe that Logic Star AI (the top-performing agent on the SWT-Bench Verified leaderboard) resolves 39 issues that no other agent resolves (hereafter referred to as case 1). Conversely, 42 issues were resolved by at least one other agent that Logic Star AI failed to resolve (hereafter referred to as case 2). Upon comparing the labeled features of the issues in these two cases, we observe that, for case 1, only 2 of the 39 (5.12%) issues contain solution-related features (i.e., two issues contain `s_nl` or `s_code`). In contrast, approximately 24% of the issues in case 2 contain solution-related features (i.e., 10 issues contain `s_nl` or `s_code`). This aligns with our finding that `s_nl` is the most important feature overall and it does not help Logic Star AI (and is negatively correlated).

Also, 6.4% more issues (56.4% case 1 vs 50.0% case 2) contain localization information (`s_loc`). It is the most important feature for Logic Star AI, while also being important for

Table 3.4: Feature importance, derived from logistic regression coefficients, is presented for the selected agents on the SWT-Bench Verified and Lite datasets. Positive coefficients indicate a positive correlation, while negative coefficients indicate a negative correlation. Statistical significance is denoted by ** (p-value ≤ 0.02) and * (p-value $\in (0.02, 0.05]$)

Agent	ic_nl	ic_eg	c_nl	c_eg	s_nl	s_code	s_loc
SWT-BENCH VERIFIED							
Logic Star AI	-0.216	0.574	-0.044	-0.361	-0.176	-0.411	0.665**
OpenHands GPT-5-mini	-1.540	-0.347	-0.230	-0.192	-0.003	-0.173	0.288
e-Otter++ Claude 3.7	0.258	0.237	-0.065	-0.524*	0.398	0.481	0.345
Amazon Q Developer	-0.241	0.564*	0.031	-0.085	0.452	-0.036	0.254
AssertFlip GPT-4o	-1.227	0.003	-0.001	0.116	0.023	-0.021	0.471*
Otter++ GPT-4o	0.053	0.302	0.205	0.075	0.130	-0.063	0.379
Otter GPT-4o	-0.105	0.291	0.289	0.031	0.023	-0.008	0.329
OpenHands Claude 3.5	0.691	0.369	0.325	-0.043	0.085	-0.460	0.373
SWT-BENCH LITE							
e-Otter++ Claude 3.7	0.339	0.466	0.160	-0.508	0.682	0.581	-0.028
Amazon Q Developer	0.863	0.634	0.327	-0.052	0.819*	0.679	0.233
AssertFlip GPT-4o	0.556	0.331	-0.120	-0.441	0.003	0.371	0.267

the other agents. We see a decrease in the example related features in case 2 compared to case 1. The presence of `ic_eg` helps Logic Star AI (in top 3 most important feature), while it is not that important overall for the other agents, hence the decrease for case 2 aligns with our observations.

We also see an increase in the presence of `c_nl` in case 2 compared to case 1. This aligns with the observation that, while important, `c_nl` is negatively correlated with performance for Logic Star AI, while being an important feature overall for the other agents.

Logic Star AI favors the presence of localization information (`s_loc`) and examples (`ic_eg`), while favoring the absence of solution-related features (`s_nl` and `s_code`) and correct behavior in natural language (`c_nl`).

We also observe that the issue `django__django-10999`¹ was resolved only by Otter++ GPT-4o and Otter GPT-4o (ranked third-last and second-last on the leaderboard, respec-

¹[Link to the issue django__django-10999 from SWT-Bench Verified](#)

tively), while none of the other agents succeeded. Upon inspection, the issue appears to contain all features except examples. It is relatively straightforward, as it clearly specifies both the location of the bug and the solution, making the generation of bug-reproducing tests comparatively simple. We speculate that other agents may have overestimated the complexity of the issue and engaged in unnecessary reasoning, whereas Otter was able to directly leverage the provided bug localization and solution information to generate appropriate bug-reproducing test cases and successfully resolve the issue.

We also observe that the agent ranked last on the leaderboard (OpenHands Claude Sonnet 3.5) was able to resolve two issues that no other agent successfully resolved. Upon inspecting these issues (`sphinx-doc__sphinx-9461`² and `django__django-13837`³), we find that they involve meta-programming and environment-dependent issues rather than conventional logical bugs. Specifically, these issues depend on Python runtime data, require specific execution environments, and are difficult to reproduce using typical unit test heuristics. We speculate that the weakest agent may have succeeded because it was able to rely more directly on hints provided in the issue descriptions and its underlying Python knowledge, rather than depending heavily on complex reasoning or elaborate testing strategies. In contrast, other agents may have attempted more sophisticated reasoning and test generation strategies, potentially over-complicating the task.

There exist relatively simple issues that even the top-performing agents were unable to resolve but were successfully resolved by agents with lower overall success rates. This observation highlights the complementary strengths of different agents and suggests that variations in their underlying LLMs and architectures can lead to diverse problem-solving capabilities.

3.4 Discussion

3.4.1 Suggestions

- When writing bug reports, the author should, whenever possible, describe the solution, provide localization details, and clearly specify the expected behavior (all in natural language).

²[Link to the issue `sphinx-doc\_\_sphinx-9461` from SWT-Bench Verified](#)

³[Link to the issue `django\_\_django-13837` from SWT-Bench Verified](#)

- Bug reports should ideally be triaged by a software engineer before being forwarded to an agent responsible for generating bug-reproducing tests. An engineer familiar with the codebase can enhance the report by providing the solution to the bug, localizing it, and correcting any errors in the original bug report. Alternatively, a fine-tuning strategy or the incorporation of relevant metadata could allow separate specialized agents, distinct from the test-generation agents, to perform these enhancements automatically.
- Consider decomposing complex bugs or feature requests into multiple, smaller bug reports to better align with the current capabilities of agents. Additionally, provide sufficient context, particularly for specialized or less familiar repositories, to enable agents to generate more accurate and effective bug-reproducing tests.
- We observe that each agent has its own strengths, and even lower-performing agents can successfully resolve issues that others fail to address. Developing an ensemble strategy that leverages models with diverse architectures or LLMs, and differing feature priorities, can more effectively tackle a broader range of issues.

3.4.2 Threats to Validity

Threats to Internal Validity

Our study may be affected by subjectivity in feature labeling, as the features were manually annotated based on issue descriptions. This introduces potential ambiguity in interpreting issue text, possible inconsistencies in labeling, and confirmation bias during the annotation process. We mitigate these threats through random spot-check validation by other authors, the use of predefined annotation guidelines, and a well-defined feature taxonomy.

Threats to External Validity

Our analysis may have limited generalizability, as it focuses on 433 issues from SWT-Bench Verified and 276 issues from SWT-Bench Lite, which may not represent other programming languages, domains, or project types. However, SWT-Bench includes 12 diverse and widely used Python repositories, which strengthens the practical relevance of our findings. As future benchmarks expand to additional languages and project settings, our analysis can be extended accordingly. Furthermore, the leaderboard reflects the SOTA agents at the time of data collection, supporting the timeliness of our results.

3.5 Conclusion

In this paper, we identify seven key features of bug reports and manually annotate 709 issues from the SWT-Bench Verified and Lite datasets. We analyze the performance of selected agents and examine the overlaps in the issues they resolve. We further investigate differences in the presence and absence of these features between issues uniquely resolved by the top-ranked agent on the leaderboard and issues that the top agent fails to resolve but that are successfully addressed by at least one other agent. In addition, we qualitatively analyze issues resolved by agents at the bottom of the leaderboard but not by other agents. Furthermore, using random forest, logistic regression and Pearson’s correlation coefficient, we identify which features most significantly influence the generation of bug-reproducing tests.

Our results show that agents exhibit complementary strengths, with each agent capable of resolving issues that others cannot. In some cases, top-performing agents may fail to resolve relatively straightforward issues. Moreover, the presence of solution in natural language, localization information and correct behavior in natural language in bug reports plays the most significant role in the test-generation performance of agents. This may vary depending on specific agent and LLM architectures. Software engineers and tool designers can leverage these insights to enhance issue triaging and build more effective, and resilient agent architectures.

Chapter 4

Conclusion

This thesis highlights the impact of different types of input on LLM and agent-based test generation. In particular, it highlights the importance of natural language requirements, solutions (when available), and localization information, and demonstrates how these factors influence the performance of LLMs and software development agents.

The conclusions from Chapter 2 show that requirements (natural language descriptions of desired outcomes) provide a powerful and widely applicable way to unlock greater value from LLMs. However, overly rigid input/output formatting requirements can degrade test quality for smaller models rather than improving it. It is also critical to select models carefully, balancing the trade-offs between smaller and larger models, and to critically evaluate model outputs rather than relying on them blindly.

The conclusions from Chapter 3 show that natural-language solutions, localization information, and descriptions of correct behavior are associated with improved performance in bug-reproducing test generation. Additionally, different agents exhibit complementary strengths, with each agent capable of addressing scenarios that others (even higher-performing ones) may struggle with.

Overall, our findings emphasize that while different LLMs and agents may have varying input preferences depending on their architectures, higher success rates are consistently observed when natural language descriptions of requirements or correct behavior, solutions, and localization information are included in the prompt or context. These insights can help readers make more effective use of existing models and agents, while also informing designers of key considerations when architecting future LLMs and agent-based systems.

4.1 Opportunities for Future Research

This thesis highlights several opportunities for future research in the domain of agent/LLM based code and test generation.

- The findings of this work could be leveraged to train or fine-tune an agent or LLM specifically for generating bug-detecting tests, potentially outperforming existing commercial tools that may instead validate the given code.
- Developing and testing an agent that leverages richer sources of information, such as design documents, meeting notes, and other project artifacts, beyond traditional requirements. Combined with appropriate tooling, such an agent could iteratively refine its responses and potentially approach or surpass human-level performance in test generation.
- Improving the performance of smaller models through techniques such as model distillation and fine-tuning could enable them to more effectively leverage requirements for test generation or infer them when absent. This approach has the potential to narrow the performance gap between smaller and larger models. The evaluation framework proposed in Chapter 2 could be reused to assess these improvements, or the agents could be evaluated using established benchmarks such as SWT-Bench.

References

- [1] Amazon Q. <https://aws.amazon.com/q/>.
- [2] Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>.
- [3] Introduction. <https://docs.openhands.dev/overview/introduction>.
- [4] LogicStar.ai | Self-Healing Applications. <https://logicstar.ai/blog/swe-bench-verified-september-16>.
- [5] SWT-Bench: Assessing capabilities at Unit Test Generation. <https://swtbench.com/>.
- [6] Writing tests with GitHub Copilot. <https://docs-internal.github.com/en/copilot/tutorials/write-tests>.
- [7] Eth-sri/SWT-bench_Lite_bm25_27k_zsb · Datasets at Hugging Face. https://huggingface.co/datasets/eth-sri/SWT-bench_Lite_bm25_27k_zsb, August 2025.
- [8] Eth-sri/SWT-bench_Verified_bm25_27k_zsb · Datasets at Hugging Face. https://huggingface.co/datasets/eth-sri/SWT-bench_Verified_bm25_27k_zsb, August 2025.
- [9] Princeton-nlp/SWE-bench_Lite · Datasets at Hugging Face. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite, March 2025.
- [10] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: Generating Tests from Issues to Validate SWE Patches, February 2025.

- [11] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection, January 2026.
- [12] Zhi Chen and Lingxiao Jiang. Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios, December 2024.
- [13] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. A Survey on Code Generation with LLM-based Agents, September 2025.
- [14] Andre Hora and Gordon Fraser. Understanding Bug-Reproducing Tests: A First Empirical Study, February 2026.
- [15] Dong Huang, Jie M. Zhang, Mark Harman, Mingzhe Du, and Heming Cui. Measuring the influence of incorrect code on test generation. *arXiv preprint arXiv:2409.09464*, 2025.
- [16] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *International Conference on Software Engineering*, pages 435–445, 2014.
- [17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, November 2024.
- [18] Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests, January 2026.
- [19] Divya Kumar and Krishn Kumar Mishra. The impacts of test automation on software’s cost, quality and time to market. *Procedia Computer Science*, 79:8–15, 2016.
- [20] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Co-daMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *International Conference on Software Engineering*, pages 919–931, 2023.
- [21] Gabrielle Lima, Diógenes Silva, Carlos Mar, and Diego Coronel. Automated Test Case Generation in a Real-World System Using a Customized AI Agent: An Experience Report. In *Simpósio Brasileiro de Qualidade de Software (SBQS)*, pages 376–385. SBC, November 2025.

- [22] Noble Saji Mathews and Meiyappan Nagappan. Design choices made by LLM-based test generators prevent them from finding bugs. *arXiv preprint arXiv:2412.14137*, 2024.
- [23] Noble Saji Mathews and Meiyappan Nagappan. Test-driven development and LLM-based code generation. In *Automated Software Engineering*, pages 1583–1594, 2024.
- [24] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents, February 2025.
- [25] Pengyu Nie, Rahul Banerjee, Junyi Jessy Li, Raymond J Mooney, and Milos Gligoric. Learning deep semantics for test completion. In *International Conference on Software Engineering*, pages 2111–2123, 2023.
- [26] Juan Altmayer Pizzorno and Emery D Berger. CoverUp: Coverage-guided LLM-based test generation. *arXiv preprint arXiv:2403.16218*, 2024.
- [27] Meghana Puvvadi, Sai Kumar Arava, Adarsh Santoria, Sesha Sai Prasanna Chen-nupati, and Harsha Vardhan Puvvadi. Coding agents: A comprehensive survey of automated bug fixing systems and benchmarks. In *2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT)*, pages 680–686, 2025.
- [28] Md Sydur Rahman, Aditya Kumar Saha, Uma Chakraborty, Humaira Tabassum Su-jana, and S. M. Abdullah Shafi. Evaluating the impact of Test-Driven Development on Software Quality Enhancement. *International Journal of Mathematical Sciences and Computing*, 10(3):51.
- [29] Ahmed Ramadan, Husam Yasin, and Burhan Pektas. The role of artificial intelligence and machine learning in software testing. *arXiv preprint arXiv:2409.02693*, 2024.
- [30] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *Transactions on Software Engineering*, 50(1):85–105, 2023.
- [31] Qodo Team. Writing Unit Tests Best Practices. <https://www.qodo.ai/blog/best-practices-for-writing-unit-tests/>, April 2023.
- [32] Hyrum Titus Winters Wright, Tom Manshreck. *Software Engineering at Google*.
- [33] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2):67–120, 2012.

- [34] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. What makes a good bug report? *IEEE Transactions on Software Engineering*, 36(5):618–643, 2010.